

When Pre-Training Meets Contrast Learning: Few-Shot Encrypted Traffic Classification With Novelty Detection

Lixin Zhao^{ID}, Changhao Wu, and Aimin Yu^{ID}

Abstract—Encrypted traffic classification plays an important role on network security and network management. However, existing methods face two key limitations: they require large volumes of labeled data and are typically designed under the closed-world assumption. This paper presents a novel approach combining pre-training techniques with contrast learning to address the challenges of open-world encrypted traffic classification. Our method utilizes a BERT-based pre-training model to learn generic traffic representations from large-scale unlabeled encrypted traffic. In this stage, two pre-training tasks, the Masked BURST Model (MBM) and BURST Context Prediction (BCP), are introduced to capture both structural and contextual features of the traffic. Following this, a contrast learning strategy is applied on a small labeled dataset to learn a discriminative feature space, where traffic from the same category is clustered closely, and traffic from different categories is largely separated. This strong discriminative property enables high-accuracy classification of known traffic categories while detecting novel ones. Experimental results on five encrypted traffic classification tasks demonstrate that: 1) under full data conditions, our approach outperforms state-of-the-art methods with up to an 11.60% improvement in F1 score in both closed- and open-world scenarios; 2) even when working in a few-shot mode, our method achieves the best performance in three out of five tasks in the closed-world setting and consistently outperforms the best methods by up to 9.99% in terms of F1 score in the open-world scenario.

Index Terms—Encrypted traffic classification, pre-training, contrast learning, open-world classification.

I. INTRODUCTION

NETWORK traffic classification, which identifies traffic types originating from different applications and web services, serves as a cornerstone for managing and securing networks. It underpins critical functions such as intrusion detection [11], quality of service (QoS) optimization [36], user experience enhancement [9], and overall network visibility [1]. However, the widespread adoption of encryption (e.g.,

TLS 1.3, VPN, Tor) also allows malware and unauthorized services to evade detection, presenting significant challenges to traditional classification methods.

To address the issue of encrypted traffic classification, research has shifted from early machine learning (ML) with handcrafted features [4], [31] [32], [33], [42] to deep learning (DL) models that automatically extract patterns from raw traffic [26], [38] [39]. While effective, DL models typically require massive labeled datasets, which are difficult to obtain due to the need for expert knowledge and the rapid evolution of traffic patterns.

In recent years, pre-training techniques have gained great success on the fields of natural language processing [8] and computer vision [10]. The pre-training based methods usually use unlabeled data to learn general knowledge in a self-supervised learning way and then fine-tuning on limited amount of labeled data to adapt the specific down-stream tasks. Some attempts, such as PERT [18], ET-BERT [25], and YaTC [51] etc. have been made in traffic classification based on pre-training techniques. However, these existing works only explore the traffic representation ability provided by pre-training techniques in closed-world setting, that means all the categories of the downstream tasks are known and fixed. As a matter of fact, due to the openness of the real world, new encrypted traffic categories will constantly emerge (e.g. newly released applications with traffic encryption technology). Therefore, in order to accommodate the real-world scenarios, the encrypted traffic classification system should be (1) built on limited labeled traffic data (i.e., few-shot condition) and (2) performs high-precision classification for known traffic categories and (3) has the ability to detect novel traffic categories.

Existing works have explored few-shot learning and novel class detection, such as FC-Net [46] and TrafficGPT [15], but address only isolated aspects of network traffic analysis. FC-Net targets few-shot classification for intrusion detection in closed-world settings, leveraging small samples but failing to handle unseen classes, limiting its applicability to dynamic network environments where new applications or threats emerge frequently. Similarly, TrafficGPT leverages GPT-2 for open-world encrypted traffic classification, demonstrating improved generalization to unknown traffic types. However, its effectiveness depends on large-scale training data and lacks dedicated few-shot learning mechanisms. Thus, the primary motivation of our work is to address the dual challenge of classifying encrypted traffic with limited labeled data (few-shot) while detecting unseen classes in open-world scenarios, a combination not fully addressed by existing methods.

Received 13 February 2025; revised 14 September 2025 and 15 January 2026; accepted 12 March 2026; approved by IEEE TRANSACTIONS ON NETWORKING Editor S. Magnusson. Date of publication 16 March 2026; date of current version 1 April 2026. This work was supported by the Project of Cloud Information Security Protection Technology under Grant E4GZ030302. (Corresponding author: Aimin Yu.)

Lixin Zhao and Aimin Yu are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China (e-mail: zhaolixin@iie.ac.cn; yuaimin@iie.ac.cn).

Changhao Wu is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: wuchanghao@iie.ac.cn).

Digital Object Identifier 10.1109/TON.2026.3674624

2998-4157 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

The core insight of our work is that effective few-shot open-world encrypted traffic classification necessitates a *protocol-aware hierarchical representation*, rather than treating traffic as generic byte streams. Unlike prior Transformer-based approaches that treat traffic as flat token sequences (e.g., ET-BERT [25]), we explicitly model the inherent hierarchical structure of network traffic. In protocols such as TLS 1.3 or complex tunneling schemes (VPN/Tor), semantically meaningful units (BURSTs, defined as consecutive packets in the same direction) naturally correspond to application-layer request/response transactions. This structural prior acts as a critical inductive bias, enabling robust generalization under few-shot and open-world conditions.

To materialize this insight, we introduce a synergistic pre-training framework centered on the BURST concept. Our framework jointly optimizes two complementary objectives: the Masked BURST Model (MBM) task, which captures intra-BURST syntactic patterns, and the BURST Context Prediction (BCP) task, which learns inter-BURST semantic relationships. This hierarchical modeling strategy yields a robust representation foundation that serves as a strong initialization for downstream adaptation. Furthermore, to address the theoretical gap in open-world traffic classification, we refine these representations through a theoretically grounded supervised contrastive learning (SCL) stage. As analyzed in Proposition 1, this explicitly optimizes the intra-class compactness and inter-class separation in the embedding space, thereby minimizing the open-world risk bound.

We evaluate the performance of our approach on five encrypted traffic classification tasks. Experimental results demonstrate that: (1) under full data conditions, our approach outperforms existing methods across all five tasks, achieving improvements of 11.60%, 0.92%, 8.83%, 5.60% and 2.16% in F1 score in the closed-world scenario, and 11.74%, 6.44%, 8.04%, 15.96% and 13.91% in the open-world scenario; (2) in the few-shot setting, our method achieves the best performance on the GEAC, ETCV, and EACT tasks, with only a marginal gap compared to state-of-the-art methods, which operate under full data conditions, on the EMC and EAC-1.3 tasks in the closed-world scenario. Notably, compared to specialized few-shot traffic classification approaches, our method yields a maximum improvement of 49.32% across all five tasks. Furthermore, in the open-world scenario, our approach (i.e., in few-shot condition) consistently outperforms the best existing methods (i.e., in full data condition) by 3.22%, 2.22%, 0.50%, 9.99% and 0.69% respectively in F1 score.

In summary, the contributions of this paper are as follows:

- **Integrated Framework for Open-World Analysis:** We explore the synergy between pre-training techniques and contrast learning to address the dual challenges of few-shot classification and novel class detection in encrypted traffic.
- **Hierarchical Pre-training Scheme:** We design two domain-specific pre-training tasks to capture intra-BURST structural patterns and inter-BURST contextual logic, respectively. Their joint optimization yields a robust foundation for few-shot adaptation.
- **Optimized Metric Space via SCL:** We customize a supervised contrastive learning (SCL) strategy that explicitly enforces intra-class compactness and inter-class

separation, which is theoretically justified to reduce the open-world classification risk.

- **Theoretical Foundation:** We provide a theoretical analysis (Proposition 1) that quantifies how the quality of the learned embedding space bounds the performance of the nearest mean novelty detector, offering insights into the method's effectiveness.
- **Extensive Validation:** We conduct comprehensive evaluations across five encrypted traffic classification tasks. Results show that our approach consistently outperforms state-of-the-art methods in both closed-world and open-world scenarios, particularly under few-shot constraints.

II. RELATED WORK

A. Encrypted Traffic Classification

Existing encrypted traffic classification methods can be divided into two main categories: ML-based methods and DL-based methods. Table I provides a summary and comparison of existing traffic classification methods, with detailed descriptions presented in the following section.

ML-based methods. The dominant ML-based methods [4], [31] [32], [33], [42] commonly employ algorithms such as SVM, KNN, and Naive Bayes to classify encrypted traffic. They typically rely on manually extracted features, such as packet count, packet size, and packet inter-arrival time etc. Recent work like [47] employs advanced handcrafted path signature features for encrypted traffic analysis, demonstrating the ongoing potential of feature engineering. The performance of these approaches hinges on the ability to extract discriminative features that effectively capture the differences among traffic categories. However, this feature extraction process is heavily dependent on domain expertise, posing challenges to the scalability and adaptability. Whisper [12] detects malicious traffic, including zero-day attacks, by leveraging frequency domain features. However, from a traffic classification perspective, while it can identify novel classes, its capability is limited to binary classification rather than fine-grained multi-class classification. FOSS [23] employs a tree-based framework for fine-grained unknown attack detection in open-world network intrusion scenarios, but it cannot effectively adapt to few-shot scenarios.

DL-based methods. DL-based methods automatically learn complex feature patterns from the raw traffic and perform classification. Multiple deep learning models are adopted by different studies, including CNN-based models [29], [38], [40], RNN-based models [26], [41] and GNN-based models [20], [39]. Besides, some works [7], [28], [50] also solve the problem of open-world traffic classification based on deep learning techniques. Although DL-based methods eliminate tedious feature engineering, they depend on a large amount of labeled traffic data for model training to learn valid features. However, it is time-consuming and error-prone to perform large-scale traffic labeling. It should be noted that some DL-based methods [46], [48], [49] focus on few-shot traffic classification, a scenario that aligns with our focus. However, these studies lack novelty detection ability. In contrast, our approach performs open-world traffic classification without requiring large amounts of labeled data.

TABLE I
COMPARISON OF THE EXISTING NETWORK TRAFFIC CLASSIFICATION METHODS

Category of Methods	Technique Roadmap	Multi-class Classification	Feature Generality	Few-shot Condition	Novelty Detection
ML-based	Statistics-related features [32],[42]	Yes	No	No	No
	Frequency domain analysis [12]	No	No	No	Yes
	Graph propagation [23]	Yes	No	No	Yes
DL-based	Meta-learning [46], [48]	Yes	Yes	Yes	No
	Pre-training [18],[25],[6],[54]	Yes	Yes	No	No
	Transfer learning [49]	Yes	Yes	Yes	No
	One-class learning [53]	Yes	No	No	Yes
	ours	Yes	Yes	Yes	Yes

B. Pre-Training Techniques

Pre-training, also known as self-supervised pre-training, is an approach that involves training a model on a large amount of unlabeled data before fine-tuning it on a specific downstream task with labeled data. Pre-trained models, such as BERT [8] and its variants [21], [27], [37], have demonstrated a powerful ability to capture generic patterns from large-scale unlabeled data, significantly enhancing performance in downstream tasks. In the field of traffic classification, PERT [18] was the first to adapt ALBERT for the task of encrypted traffic classification, demonstrating promising performance. However, it lacks a tailored design for encrypted traffic representation and pre-training tasks, which constrains its generalization to emerging encryption technologies. To address these limitations, ET-BERT [25] introduces a novel representation method for encrypted traffic and redesigns two pre-training tasks. Similarly, TrafficFormer [54] develops a more fine-grained and complex pre-training task, termed the Same Origin-Direction-Flow (SODF) task. TrafficLLM [6] introduces a dual-stage tuning framework leveraging LLMs for network traffic analysis, using traffic-domain tokenization along with EA-PEFT to enhance detection and generation tasks. However, all these methods require extensive fine-tuning data and are confined to closed-world assumptions, failing to generalize to novel classes outside the training distribution. For open-world scenarios, OWCP [24] synthesizes new classes in a spherical space for N+1 classification, yet risks overfitting due to the complexity of sampling from an infinite class space. TrafficGPT [15] utilizes GPT-2 [35] combined with the K-LND [7] distance metric for novelty detection. However, as demonstrated in Section V-E, generic pre-trained features without explicit discriminative constraints often lack sufficient class separability. Unlike these approaches, we explicitly impose a supervised metric learning objective to enforce a compact and discriminative manifold, which is critical for robust few-shot and open-world classification.

C. Contrast Learning

Contrast learning is a key approach in self-supervised representation learning, aiming to bring similar samples closer in feature space while pushing dissimilar ones apart. While it has achieved significant success in computer vision (e.g. SimCLR [5], MoCo [19], BYOL [17]) and natural language processing

(e.g. SimCSE [13]), applying it to network traffic remains challenging due to the multi-source heterogeneity and semantic ambiguity of raw traffic data. In this paper, we bridge pre-training with contrast learning to better characterize encrypted traffic. Unlike self-supervised paradigms such as SimCLR or MoCo, which rely on data augmentations and lack explicit label guidance, our contrastive learning strategy is based on the Supervised Contrastive Learning (SCL) framework. Moreover, existing studies mainly adopt contrastive learning in a heuristic manner, without offering theoretical guarantees in open-world settings. This limitation motivates our formulation of SCL tailored for encrypted traffic, which explicitly optimizes discriminability under few-shot and novelty-aware constraints.

III. METHODOLOGY

A. Model Overview

The overall framework of our approach is shown in Fig. 1, which consists of two stages: pre-training and contrast learning. The goal of pre-training is to learn generic patterns for each traffic flow from large-scale unlabeled encrypted traffic, and the pre-trained model is then used as a feature extractor to obtain high-quality initial feature vector representations to conduct contrast learning. In the contrast learning stage, given the limited labeled traffic data, it performs similarity learning under the supervision of a customized contrastive loss function to obtain an embedding space, in where an open-world nearest mean classifier is applied.

B. Pre-Training Stage

In the pre-training stage, the model is trained on a massive unlabeled traffic dataset. The raw traffic data is first converted into tokens (tokenization), and then constructed into the input for the pre-training tasks. There are two pre-training tasks: Masked BURST Model (MBM) task and BURST Context Prediction (BCP) task.

1) *Data Preprocessing*: Unlike natural language text which contains explicit semantic units, traffic data is a pure byte stream on the network. In order to effectively leverage the pre-training technique for generic traffic patterns learning, we perform the follow data preprocessing steps: (1) Flow-to-BURST, (2) BURST Tokenization, (3) Token Embedding.

(1) *Flow-to-BURST*: Give the raw traffic data, we first split out flows with the same 5-tuple (i.e., source IP, source port,

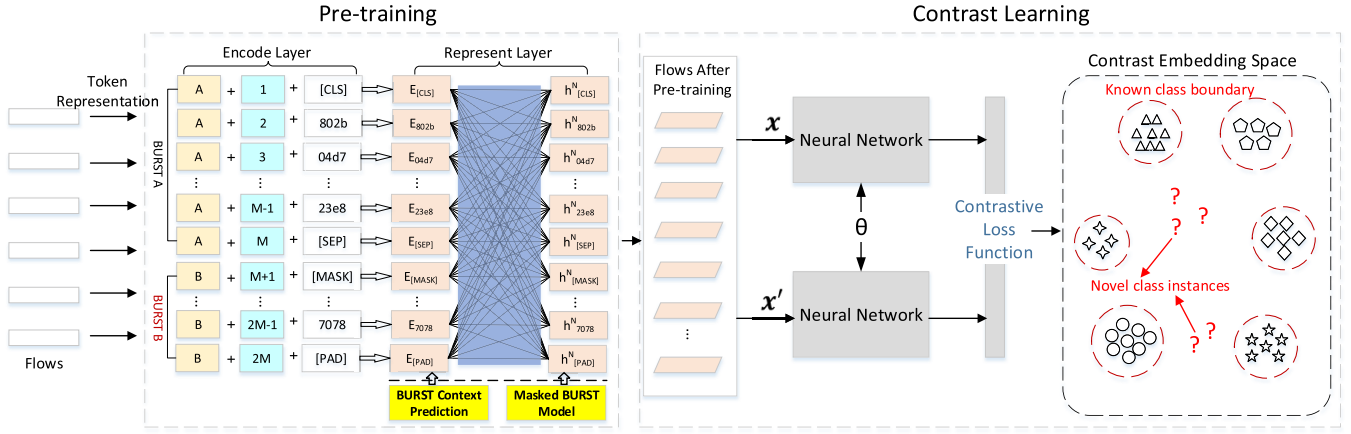


Fig. 1. Overview of the whole framework.

destination IP, destination port, and protocol), which includes packets in both directions. Then, each flow is converted into multiple BURSTs each of which contains a sequence of consecutive packets transmitted along the same direction [34]. BURST arises from application-layer behavior and underlying protocol dynamics and it typically represents a distinct logical unit of data exchange. In modern web services, the interaction between client and server often involves segmented resource requests and responses, driven by the need to fetch multiple objects such as scripts, stylesheets, or media files. These segmented transfers are shaped by the concurrency mechanisms of application-layer protocols like HTTP/2 and transport-layer features like TCP's congestion control management. As a result, a BURST reflects both the structure of application data and the temporal characteristics of its transmission, offering insights into the operational patterns of the specific application.

(2) **BURST Tokenization**: In order to transform the BURSTs into the token representations for pre-training, we draw an analogy between traffic flows and documents in natural language. From this perspective, BURSTs in flows can be mapped to sentences in documents. To transform BURSTs into tokenized representations, we decompose each BURST into a sequence of units analogous to words in natural language. Considering that BURST is composed of a string of hexadecimal numbers such as 802b04d732, a 2-gram model is employed to segment hexadecimal BURST data into units consisting of 2 adjacent bytes. For the above example, after segmentation, the results are 802b 2b04 04d7 d732. We empirically set the size of segment unit to 2 to limit the corpus size, since a large corpus space will significantly increase computational cost. These units are further tokenized using the Byte-Pair Encoding tokenizer, with a maximum corpus size constrained to 65536 tokens. In addition to the BURST units, special tokens such as [CLS], [SEP], [MASK] and [PAD] are included in the corpus. [CLS] is used for classification tasks. [SEP] is used to separate different BURSTs in the BCP task. [MASK] is used to replace the masked tokens in the MBM task. [PAD] is used to pad the input to the maximum length.

(3) **Token Embedding**: We select BERT as the base model for pre-training. Since the encoder of the BERT lacks convolutional or recurrent components, it is essential to incorporate additional information that enables the model to capture the positional relationships within the input data. Additionally, to

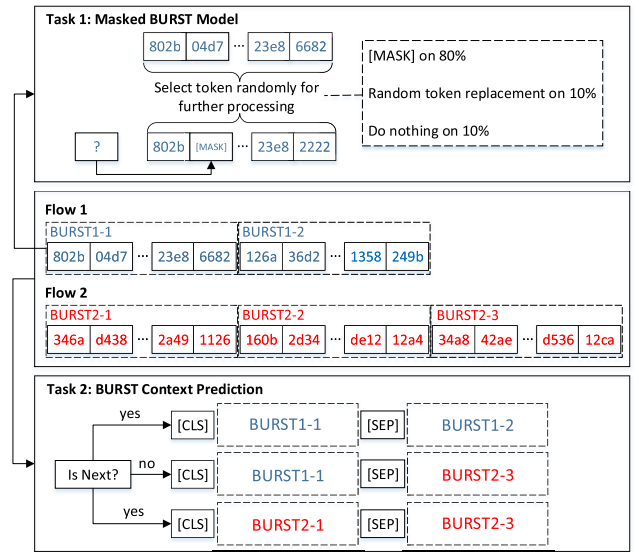


Fig. 2. Illustration of the two pre-training tasks for generic traffic feature learning.

facilitate the BCP task, it is necessary to include supplementary identifiers that encode the BURST segment information. These two steps are referred to as positional embedding and segment embedding, respectively. Furthermore, the process of transforming BURSTs into tokenized representations is known as token embedding. Collectively, these three embedding processes—positional, segment, and token embedding constitute the input representation required for pre-training.

2) **Pre-Training Tasks**: As shown in Fig. 2, we design two pre-training tasks, namely MBM task and BCP task, to effectively capture the internal relationships between bytes within a single BURST and the interrelations between BURSTs originating from the same flow or across different flows. The motivation for this two-task design is to learn a comprehensive representation of traffic flows by capturing both low-level byte patterns within individual protocol data units (BURSTs) and high-level contextual relationships between these units. These two tasks are jointly optimized to enable the model to comprehend both intra-BURST structures and inter-BURST correlations, thus facilitating the learning of more generic feature representations for each flow.

• **MBM TASK.** Similar to the Masked Language Modeling and the Masked Image Modeling introduced in BERT [8] and BEiT [2] respectively, the MBM task is utilized to learn the internal byte dependencies within BURSTs by predicting randomly masked tokens. This task forces the model to understand the syntactic structure and internal correlations of a BURST, which is often determined by specific network protocols (e.g., the structure of a TLS handshake record or an HTTP header). For example, in a BURST containing a TLS Client Hello message, the values of certain bytes (e.g., version, cipher suites, extensions) are highly correlated. If the bytes indicating the TLS version are masked, the model must infer their value based on the surrounding context bytes that define the TLS handshake structure. In detail, as shown in the upper part of Fig. 2, we first randomly select tokens to be masked in the input BURST. Then, for each of the masked tokens, one of the following actions is performed: (1) Replace it with the [MASK] token with 80% probability. (2) Replace it with another token with 10% probability. (3) Do nothing with 10% probability. Finally, a BERT-based model is trained to predict [MASK] based on the preceding and following bytes within the BURST. If there are k tokens are randomly masked in the input BURST S , the loss function for the MBM task L_{MBM} can be defined as:

$$L_{MBM} = - \sum_{i=1}^k \log(P(MASK_i = token_i | \hat{S}; \theta)) \quad (1)$$

where θ is the set of trainable parameters of Transformer encoder and P is the probability function modeled by θ . $\hat{S}$ is the representation of BURST S after masking and $MASK_i$ is the masked token at the i_{th} position in BURST S .

• **BCP TASK.** The MBM task discussed above focuses on learning intra-BURST byte dependencies within individual traffic units. However, modeling isolated BURSTs alone is insufficient to capture the generic behavioral patterns of complete traffic flows. To this end, we design the BURST Context Prediction (BCP) task to learn contextual dependencies across BURSTs at the flow level.

Unlike Next Sentence Prediction (NSP) in natural language processing or the Same-origin Burst Prediction (SBP) task adopted in prior work such as ET-BERT [25], our BCP task is explicitly grounded in the structural characteristics of network traffic. In ET-BERT, segment-level prediction is typically constructed by splitting packet sequences at predefined or arbitrary boundaries, which is effective for capturing local co-occurrence patterns but does not explicitly model how semantically meaningful transmission units interact over time. In contrast, we adopt the BURST, which naturally delimited by changes in packet direction, as the basic semantic unit. A BURST often corresponds to a complete request or response at the application layer, thereby preserving protocol-level transactional integrity.

Based on this formulation, the BCP task aims to capture higher-order, flow-level semantics by modeling whether two BURSTs are contextually related within the same communication session. Such relationships are often indicative of application behavior. For example, in a video streaming flow, a small request BURST from the client is typically followed by a large response BURST from the server

containing video data. By learning to distinguish whether two BURSTs originate from the same flow or from unrelated flows, the model is encouraged to encode inter-BURST interaction patterns, including request-response coupling, protocol handshake structure, and transactional rhythms that are invariant to payload encryption. These flow-level semantics are difficult to capture using packet-level or intra-BURST prediction objectives alone, yet they are critical for representing complex encrypted tunnels such as TLS 1.3 and Tor traffic.

Concretely, the BCP task is formulated as a binary classification problem. As illustrated in the lower part of Fig. 2, during pre-training, two BURSTs are randomly selected to form a BURST pair. These paired BURSTs are labeled as $BURST-a$ and $BURST-b$ through segment embedding. With a probability of 50%, $BURST-b$ is sampled from the same flow as $BURST-a$; otherwise, it is sampled from a different flow. Given a BURST pair $B_i = (BURST-a_i, BURST-b_i)$ and its ground-truth label $y_i \in \{0, 1\}$, where $y_i = 1$ indicates that the two BURSTs originate from the same flow, the loss function of the BCP task is defined as:

$$L_{BCP} = - \sum_{i=1}^n \log(P(1 - y_i | B_i; \theta)) \quad (2)$$

In the end, as shown in Equation 3, a weighted sum of L_{MBM} and L_{BCP} is used as the final loss function for pre-training. Here, λ is a hyperparameter for balancing the loss values of the two tasks.

$$L = \lambda * L_{MBM} + L_{BCP} \quad (3)$$

In essence, the MBM task learns the syntax of traffic (the structure of individual BURSTs), while the BCP task learns the storyline (how BURSTs logically connect to form a meaningful flow). Their combination provides a robust and general-purpose representation for encrypted traffic flows.

C. Contrast Learning Stage

In traditional methods based on pre-training techniques, they usually perform fine-tuning by first initializing with the parameters of the pre-trained model and then continue to train on the data of the specific task. Unlike these methods, we use the trained model as a feature extractor to get robust initial feature vector representations for traffic flows and these feature vectors are further used as the input for contrast learning. This decoupled two-stage formulation provides superior scalability in practical deployment, because it avoids repeatedly fine-tuning the large pre-trained backbone and enables efficient updates through retraining only the small contrastive module when new classes emerge. To be more specific, when the pre-training stage is finished, the parameters are frozen and the raw traffic data for contrast learning are processed into the same form as the pre-training stage. Through multi-layer nonlinear transformation, we obtain the last hidden state as inputs of the contrast learning stage.

1) *Contrast Learning Strategy:* Based on the feature representations output from the pre-trained model, this contrast learning module aims at learning a contrast embedding space where instances (i.e. traffic flows) have small intra-class scatter and big inter-class separation. To this end, give each paired training instances ($\hat{x}_i = f(x_i; \theta)$, $\hat{x}_j = f(x_j; \theta)$), where x_i and

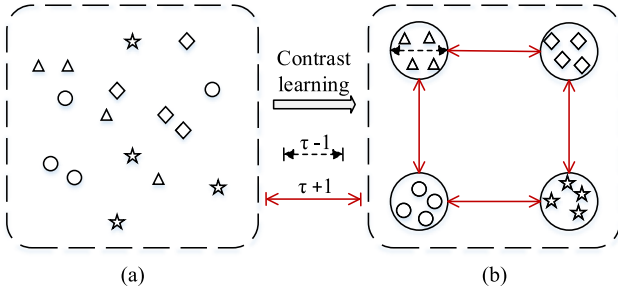


Fig. 3. Illustration of our proposed contrast learning strategy. (a) original feature space. (b) contrast embedding space.

x_j denote raw traffic flows after preprocessing and $f(\cdot; \theta)$ is a nonlinear mapping modeled by the pre-trained model, the pair-wise feature distance metric of $(\hat{x}_i, \hat{x}_j)$ can be measured by computing the Euclidean distance between their feature representations in the contrast embedding space, which can be defined as follows:

$$D(\hat{x}_i, \hat{x}_j) = \|g(\hat{x}_i; \hat{\theta}) - g(\hat{x}_j; \hat{\theta})\|_2 \quad (4)$$

where $g(\cdot; \hat{\theta})$ is modeled by a neural network.

In order to obtain more discriminative feature representations in the contrast embedding space, we expect that the distances between similar pairs are smaller than those between dissimilar pairs and there is a large margin between the similar pairs and dissimilar ones. In our supervised contrast learning paradigm, the similar and dissimilar pairs are constructed according to the traffic classes in the downstream tasks. Specifically, if two instances belong to the same traffic class, they will be considered a similar pair; otherwise, they will be considered a dissimilar pair. In addition, to avoid the problem of data imbalance, we downsample dissimilar pairs to match the number of similar pairs. In this regard, if $\hat{x}_i$ and $\hat{x}_j$ are from the same traffic class, their contrast embedding feature distance $D(\hat{x}_i, \hat{x}_j)$ should be smaller than an up-margin τ_1 ; if $\hat{x}_i$ and $\hat{x}_j$ are from different traffic classes, $D(\hat{x}_i, \hat{x}_j)$ should be bigger than a down-margin τ_2 . This can be formulated as:

$$\begin{cases} D^2(\hat{x}_i, \hat{x}_j) < \tau_1, y_i = y_j \\ D^2(\hat{x}_i, \hat{x}_j) > \tau_2, y_i \neq y_j \end{cases} \quad (5)$$

where τ_2 should be bigger than τ_1 to enforce the discriminative feature learning. Furthermore, we introduce an intermediate parameter τ to reduce the number of free parameters. Specifically, we set $\tau_1 = \tau - 1$ and $\tau_2 = \tau + 1$, thus the constraint in (5) can be simplified as:

$$1 - y_{ij}(\tau - D^2(\hat{x}_i, \hat{x}_j)) < 0 \quad (6)$$

where y_{ij} denotes whether the paired instances $(\hat{x}_i, \hat{x}_j)$ come from the same class, which is defined as follows:

$$y_{ij} = \begin{cases} 1, y_i = y_j \\ -1, y_i \neq y_j \end{cases} \quad (7)$$

An intuitive illustration of our proposed contrast learning strategy is shown in Fig. 3. It can be seen that, with the constrain in (6), there will be an explicit margin between the similar and dissimilar pairs in the learned contrast embedding space. Through imposing this constrain on each similar and

dissimilar pair in the training set, we can define the loss function for contrast learning as follows:

$$L_{CL} = \sum_{i,j} h(1 - y_{ij}(\tau - \|g(\hat{x}_i; \hat{\theta}) - g(\hat{x}_j; \hat{\theta})\|_2^2)) \quad (8)$$

where $h(x) = \max(0, x)$ is the hinge loss function. It is worth noting that, thanks to the generic feature representations produced by the pre-trained model, only a small amount of instances need to be labeled (i.e. few-shot) for contrast learning to obtain a discriminative feature space which contributes to performing open-world classification.

2) *Open-World Classification*: In traditional pre-training based traffic classification methods, a classifier is commonly fine-tuned under a closed-world assumption where the number of possible labels associated with instances at evaluation stage is known and fixed. However, in real-world scenarios, this assumption does not hold. In our problem setting, instances from previously unknown categories may appear after the classifier is built. Inspired by the Nearest Mean Classifier utilized in [30], we compute a novelty class detection threshold T_i for each known class i to reject novel class's instances and classify instances belonging to known classes. Here, to determine the optimum threshold value T_i for each class i , we assume that the instances of each class i follow a normal distribution in the contrast embedding space, characterized by a specific mean value μ_i and variance value σ_i^2 . Through applying the average intra-class distance with a narrow range of float, we obtain the threshold T_i for each class i . This can be formulated as:

$$T_i = \left(\frac{1}{\|M_i\|} \sum_{\hat{x} \in M_i} \|g(\hat{x}; \hat{\theta}) - \mu_i\|_2^2 \right) + \eta \sigma_i \quad (9)$$

where M_i is the training set of class i and $\|M_i\|$ denotes the number of instances in M_i . η is a hyperparameter to control the classification boundary for each known class.

Finally, to predict the label y_i for a raw traffic flow x_i , we first transform x_i through $f(\cdot; \theta)$ to get its initial feature representation $\hat{x}_i$. Then, we further project $\hat{x}_i$ to the contrast embedding space through $g(\cdot; \hat{\theta})$. In the contrast embedding space, we compute the distance set $\bar{D}_{\hat{x}_i, j}$ from $g(\hat{x}_i; \hat{\theta})$ to the class center of each known class j . Lastly, we select the minimum distance $\min \bar{D}_{\hat{x}_i, j}$ from $\bar{D}_{\hat{x}_i, j}$ and obtain the corresponding class label y_j . If $\min \bar{D}_{\hat{x}_i, j}$ is less than the threshold T_j , the predicted label for x_i is y_j . Otherwise, x_i will be identified as an instance from novel class. This can be formulated as:

$$y_i = \begin{cases} \underset{j=1,2,\dots,C}{\operatorname{argmin}} \bar{D}_{\hat{x}_i, j}, & \text{if } \min \bar{D}_{\hat{x}_i, j} < T_j \\ -1, & \text{otherwise} \end{cases} \quad (10)$$

where C is the number of known classes and -1 denotes the label of novel class.

D. Theoretical Justification

This section provides the analytical foundation for our methodology, specifically addressing why SCL facilitates novelty detection. We formalize the open-world setting and derive a generalization bound based on class separation.

Setting. Let $\mathcal{P}_1, \dots, \mathcal{P}_C$ be the distributions of the C known classes, and $\mathcal{P}_{\text{novel}}$ the *unknown* novel class distribution. At training time we observe n labeled instances from $\bigcup_{c=1}^C \mathcal{P}_c$ only; $\mathcal{P}_{\text{novel}}$ is revealed *only at test time*. Denote by $\Sigma_{\text{in}} = \mathbb{E}_{c=1..C} \mathbb{E}_{x \sim \mathcal{P}_c} [(g(x) - \mu_c)(g(x) - \mu_c)^\top]$ the intra-class covariance, and by $\Sigma_{\text{out}} = \mathbb{E}_{c=1..C} [(\mu_c - \bar{\mu})(\mu_c - \bar{\mu})^\top]$ the inter-class covariance, where $\mu_c = \mathbb{E}_{x \sim \mathcal{P}_c} [g(x)]$ and $\bar{\mu} = \mathbb{E}_{c=1..C} [\mu_c]$.

Proposition 1: Assume $\Sigma_{\text{in}} \leq \bar{\lambda} \Sigma_{\text{out}}$ with $\bar{\lambda} < 1$. Then the expected 0-1 risk of the nearest-mean classifier h satisfies

$$\mathcal{R}(h) \leq \mathcal{R}_{\text{OW}}^* + \frac{4\bar{\lambda} \text{Tr}(\Sigma_{\text{out}})}{\tau^2} + \mathcal{O}\left(\frac{d \log n}{n}\right) \quad (11)$$

where $\mathcal{R}_{\text{OW}}^*$ is the minimum attainable risk under the above open-world protocol, d the embedding dimension, and τ the margin in (8).

Proof: For any test sample x drawn from one of the $C + 1$ distributions, define the margin

$$\gamma(x) = \min_{c' \in \{1, \dots, C, \text{novel}\}} (\|g(x) - \mu_{c'}\| - \|g(x) - \mu_c\|) \quad (12)$$

where c is the true class of x . A classification error occurs if and only if $\gamma(x) < 0$. By Chebyshev's inequality,

$$\mathbb{P}(\gamma < 0) \leq \frac{\text{Var}(\gamma)}{(\mathbb{E}\gamma)^2} \quad (13)$$

Under the covariance assumption $\Sigma_{\text{in}} \leq \bar{\lambda} \Sigma_{\text{out}}$ we have

$$\text{Var}(\gamma) \leq 4 \text{Tr}(\Sigma_{\text{in}}) \leq 4\bar{\lambda} \text{Tr}(\Sigma_{\text{out}}) \quad (14)$$

The margin loss (8) drives the embedding such that $\mathbb{E}\gamma \geq \tau$ in population. With finite instances we only achieve $\mathbb{E}\gamma \geq \tau - \varepsilon(n)$, where $\varepsilon(n) = \mathcal{O}(\sqrt{d/n})$ is the uniform-deviation bound yielded by the Rademacher complexity of the d -dimensional linear nearest-mean rule [3]. Inserting this lower bound into Chebyshev's inequality gives

$$\begin{aligned} \mathbb{P}(\gamma < 0) &\leq \frac{\text{Var}(\gamma)}{(\mathbb{E}\gamma)^2} \leq \frac{4\bar{\lambda} \text{Tr}(\Sigma_{\text{out}})}{(\tau - \varepsilon(n))^2} \\ &= \frac{4\bar{\lambda} \text{Tr}(\Sigma_{\text{out}})}{\tau^2} \left(1 + \mathcal{O}\left(\frac{\varepsilon(n)}{\tau}\right)\right) \\ &= \frac{4\bar{\lambda} \text{Tr}(\Sigma_{\text{out}})}{\tau^2} + \mathcal{O}\left(\frac{d \log n}{n}\right) \end{aligned} \quad (15)$$

where the last step uses $\varepsilon(n) = \mathcal{O}(\sqrt{d/n})$ and Taylor expansion under $\varepsilon(n) \ll \tau$. Since $\mathcal{R}(h) - \mathcal{R}_{\text{OW}}^* \leq \mathbb{P}(\gamma < 0)$, the proposition follows. $\square$

Interpretation. The MBM+BCP pre-training furnishes generic and robust initial embeddings in which contrastive learning can sculpt a highly discriminative space; empirically, this yields intra-class covariance Σ_{in} that is markedly smaller than inter-class covariance Σ_{out} . The derived bound confirms that this compactness, which captured by a small $\bar{\lambda}$, directly tightens the risk upper-bound for the nearest-mean classifier, all without requiring any novel class instances during training.

IV. EXPERIMENTAL SETUP

A. Datasets and Tasks

We conduct five encrypted traffic classification tasks based on seven public available datasets to evaluate our approach.

The overall statistics and description about the tasks and the corresponding datasets are shown in Table II. Before using these datasets, we remove packets that are irrelevant to actual data transmission, such as packets using ARP protocol and DHCP protocol. In addition, to avoid model bias caused by strong correlation information in each traffic flow, we remove the Ethernet header, the IP address from the IP header and the protocol port from the TCP/UDP header.

B. Datasets Usage Settings

• **Pre-training Dataset.** We select approximately 10GB of unlabeled traffic data from the ISCX-VPN [14] dataset for pre-training. This dataset provides network traffic captured for popular applications, including Skype, YouTube, Facebook, and others. It includes both VPN and non-VPN traffic, offering a balanced representation of encrypted and unencrypted scenarios.

• **Closed-world Settings.** To verify the effectiveness of our approach and compare with other baseline studies in closed-world scenario, each dataset is randomly divided into the training set, the validation set and the test set in a ratio of 8: 1: 1. Among them, the training set is used to conduct contrast learning and the validation set is used for optimal hyperparameter selection and the test set is used to measure the quality of classification results.

• **Open-world Settings.** Similarly, to verify the effectiveness of various methods in open-world scenario, we make the follow settings: for each dataset, 80% of the classes are randomly selected as known classes, while the remaining 20% are designated as novel classes. The instances from the known classes are further divided into the training set, the validation set, and the test set in a ratio of 8: 1: 1. The test set of the known classes combined with all instances from the novel classes constitutes the complete test set.

C. Implementation

We implemented our approach on a server equipped with one NVIDIA RTX4090-24G GPU and an AMD 7542 CPU (14 vCPUs), 112 GB of memory. The main network architecture of pre-training model consists of multiple layers of bidirectional Transformer blocks, each composed of multi-head self-attention layers [44]. In detail, the network is composed of 12 Transformer blocks, with each self-attention layer including 12 attention heads. We set the dimension of each input token to 128, and the number of input tokens is 512. In the pre-training stage, we set the balancing coefficient λ to 1. The batch size is set to 32 and the total steps is set to 500,000. The learning rate is 2×10^{-5} , and the warmup ratio is 0.1. A residual network architecture is applied for conducting contrast learning. Specifically, the network consists of four residual blocks, where each block contains 2 convolutional layers, followed by a skip connection that allows the input to bypass the convolutional operations and be added directly to the output. In the contrast learning stage, we set the batch size to 32, the total epochs to 20 and the learning rate to 0.001. In addition, the margin τ is set to 5, and the threshold adjustment coefficient η is set to 2, both of which are determined through optimal hyperparameter search on the

TABLE II
OVERVIEW OF THE TASKS AND DATASETS. THE DESCRIPTION OF EACH TASK IS DETAILED IN THE DESCRIPTION COLUMN

Task	Dataset	#Flow	#Label	Description
GEAC	Cross-Platform (iOS) [43] Cross-Platform (Android) [43]	20,858 27,846	196 215	The General Encrypted Application Classification (GEAC) task is focused on classifying application traffic that is encrypted using standard protocols. This task is conducted on two datasets which contain 196 and 215 applications respectively.
EMC	USTC-TFC [45]	9,853	20	The Encrypted Malware Classification (EMC) task focuses on classifying encrypted traffic originating from both malware and benign applications. This task is conducted on USTC-TFC dataset which contains 10 malware and 10 benign applications.
ETCV	ISCX-VPN (Service) [14] ISCX-VPN (App) [14]	3,486 2,204	5 13	The Encrypted Traffic Classification on VPN (ETCV) task aims to classify encrypted traffic transmitting over Virtual Private Networks (VPNs). This task is conducted on ISCX-VPN dataset, which includes traffic captured from VPNs. This dataset is categorized into ISCX-VPN(Service), with 5 service categories, and ISCX-VPN(App), with 13 application categories.
EACT	ISCX-Tor [22]	3,122	16	The Encrypted Application Classification on Tor (EACT) task focuses on classifying encrypted traffic transmitting over Tor network. This task is conducted on ISCX-Tor dataset, which includes traffic from 16 applications. Tor's distributed routing makes encrypted traffic classification more challenging.
EAC-1.3	CSTNET-TLS 1.3 [25]	46,372	120	The Encrypted Application Classification on TLS 1.3 (EAC-1.3) task aims to classify encrypted traffic using the TLS 1.3 protocol. This task is conducted on CSTNET-TLS 1.3 dataset, which contains traffic data from 120 websites using TLS 1.3 protocol.

TABLE III
PERFORMANCE METRICS FOR DIFFERENT TASKS WITH VARYING K-SHOTS IN CLOSED-WORLD SETTING

Tasks	GEAC				EMC				ETCV				EACT				EAC-1.3			
K	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
K=1	0.188	0.204	0.128	0.151	0.509	0.489	0.456	0.469	0.447	0.389	0.486	0.427	0.451	0.402	0.366	0.376	0.231	0.221	0.198	0.202
K=10	0.505	0.494	0.418	0.449	0.893	0.868	0.805	0.831	0.749	0.711	0.707	0.709	0.803	0.798	0.757	0.772	0.686	0.641	0.612	0.620
K=20	0.653	0.582	0.549	0.551	0.974	0.975	0.973	0.974	0.882	0.861	0.827	0.839	0.945	0.946	0.939	0.940	0.818	0.815	0.811	0.812
K=50	0.661	0.602	0.568	0.574	0.983	0.986	0.982	0.983	0.908	0.902	0.896	0.898	0.962	0.959	0.963	0.959	0.837	0.832	0.824	0.829
K=100	0.708	0.645	0.601	0.606	0.993	0.994	0.992	0.992	0.926	0.919	0.910	0.912	0.989	0.986	0.979	0.980	0.876	0.863	0.849	0.852

validation set of the EMC task. These values are fixed for all subsequent experiments.

V. EVALUATION

In this section, we evaluate our approach on five encrypted traffic classification tasks to answer the following questions:

- **RQ1:** How effective is our approach under different k-shot settings?
- **RQ2:** How does our approach compare to existing methods under both closed-world and open-world settings?
- **RQ3:** Is the proposed components effective in enhancing the overall performance?
- **RQ4:** Do the two proposed pre-training tasks contribute to learning generic and robust feature representations, rather than simply benefiting from increased model capacity?
- **RQ5:** How does our approach perform in terms of time overhead and system resource consumption?

A. Evaluation Metrics

We assess and compare the performance of the various methods through four typical metrics, including Accuracy (AC), Precision (PR), Recall (RC), and F1 score. To avoid biases caused by data imbalance, we calculate macro average for each metric. Moreover, for the GEAC task, we report the macro average results across the Cross-Platform (iOS) and Cross-Platform (Android) datasets for brevity. Similarly, for the ETCV task, we provide the macro average over the ISCX-VPN (Service) and ISCX-VPN (App) datasets.

B. K-Shot Classification

In this section, we demonstrate the performance of our approach under different *K-shot* settings. We verify it on both closed-world and open-world scenarios. Here, the symbol *K* in *K-shot* is not the absolute number of instances in each class used for training. Instead, *K* means *K%* of instances are randomly selected from each class of the training set to conduct contrast learning (e.g. 10-shot indicates that we randomly select 10% of instances from each class in training set). The results are detailed in Table III and Table IV.

Closed-world: As shown in Table III, when $K = 1$, the performance across all the five tasks is worst because too few training instances contain limited information about the diversity of encrypted traffic, which leads to the model being in an under-fitting state. As the value of *K* increases, the classification performance is becoming better and better. This is not surprising since more information of the training classes is incorporated into the training phase as *K* increases. Specifically, as *K* grows to 10, the F1 score improves to 0.449, 0.831, 0.709, 0.772 and 0.620 respectively on the five tasks. When *K* reaches 20, the classification performance can be close to the optimal results achieved in $K = 100$, with the F1 score gaps only 9.08%, 1.81%, 8.00%, 4.08% and 4.69% respectively across the five tasks. This can be attributed to the generic initial feature representations obtained from the customized pre-training model for traffic flows. These representations enable the subsequent contrast learning task to learn discriminative features even with only a few instances. The classification performance of GEAC task is relatively low (i.e., F1 score is 0.606 when $K = 100$ and 0.551 when

TABLE IV
PERFORMANCE METRICS FOR DIFFERENT TASKS WITH VARYING K-SHOTS IN OPEN-WORLD SETTING

Tasks	GEAC				EMC				ETCV				EACT				EAC-1.3			
	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
K=1	0.167	0.185	0.122	0.149	0.431	0.422	0.401	0.406	0.345	0.357	0.296	0.311	0.281	0.295	0.306	0.295	0.190	0.191	0.183	0.179
K=10	0.416	0.424	0.388	0.401	0.801	0.808	0.791	0.793	0.638	0.621	0.589	0.608	0.747	0.738	0.746	0.739	0.602	0.591	0.562	0.571
K=20	0.619	0.579	0.538	0.545	0.922	0.925	0.920	0.921	0.831	0.819	0.808	0.812	0.905	0.908	0.911	0.903	0.751	0.718	0.739	0.724
K=50	0.634	0.599	0.555	0.562	0.943	0.944	0.942	0.941	0.855	0.842	0.830	0.831	0.932	0.929	0.933	0.928	0.782	0.791	0.784	0.785
K=100	0.683	0.612	0.582	0.590	0.962	0.959	0.963	0.959	0.905	0.900	0.861	0.873	0.957	0.951	0.961	0.952	0.829	0.831	0.811	0.819

$K = 20$). Apart from the large number of categories of GEAC task, we analyze the reasons for the category similarities, which contribute to the task's difficulty, including: (1) Similar application behaviors—applications of same type (e.g. stream media applications such as Youku, Tencent Video, iQiyi etc. or social media applications such as Wechat, QQ etc.) exhibit overlapping traffic patterns and similar burst characteristics (e.g., frequent, short bursts for messaging), reducing inter-class discriminability. (2) Shared third-party domains—multiple applications connect to common third-party domains for shared services, such as googleadservices.com for advertising, appsflyer.com for mobile attribution and marketing analytics, and googleapis.com for various Google API services, further blurring category boundaries. However, although the classification accuracy is lower compared to other tasks, we still achieved the best results compared to other related works, which is discussed in section V-C.

Open-world: As shown in Table IV, compared to the closed-world setting, the classification performance has slightly decreased in the open-world setting, e.g., 2.64%, 3.33%, 4.28%, 2.86% and 3.87% decrease in F1 score respectively across the five tasks when $K = 100$. This decrease is in accordance with expectation because open-world classification requires model to obtain more discriminative features and has stronger generalization ability. Specifically, in the closed-world setting, the training set and test set both come from known categories, so the model is easy to learn the characteristics of each category and achieve good results on the test set. In comparison, in the open-world scenario, the model should have the ability to adapt to the instances of unknown categories. However, it is worth noting that our approach still achieves high classification performance across the five tasks when K reaches 20 (i.e., F1 scores of 0.545, 0.921, 0.812, 0.903, 0.724) and it achieves the best results with F1 scores of 0.590, 0.959, 0.873, 0.952 and 0.819 when $K = 100$. To demonstrate the advantage of our approach for few-shot encrypted traffic classification, we will mainly compare the results obtained at $K = 20$ with baselines in the following sections.

C. Comparison Under Closed-World Setting

For the comparison under the closed-world setting, we evaluate a diverse set of baselines, comprising two ML-based methods: AppScanner [42] and BIND [32], two DL-based methods: FS-Net [26] and Deeppacket [29], and four pre-training based methods: PERT [18], ET-BERT [25], TrafficFormer [54], and TrafficLLM [6]. Since these methods are not optimized for few-shot learning, we train them on

the entire training set for each task to demonstrate their best performance. For pre-training based approaches, we adhere to the dataset selection protocols outlined in their respective papers. Additionally, we incorporate three recent few-shot learning methods—FC-Net [46], MetaMRE [48], and FS-MTC [49]—evaluated under a consistent 20-shot condition for fair comparison. The experimental results are shown in Table V.

• **GEAC Task.** The results for the GEAC task are shown in the leftmost part of Table V. We can observe that the pre-training based methods are consistently better than both ML-based and DL-based methods. The reason is that the pre-training based methods obtain essential packet or flow features from large-scale unlabeled traffic data which contributes a lot to the built of target classifier. Among all pre-training based methods, PERT has the worst performance since it has not designed specific task to learn the transmission structure of traffic. ET-BERT and TrafficFormer perform better than PERT by respectively introducing well-designed pre-training task for traffic feature learning. TrafficLLM, which uses LLMs for generic traffic representation, achieves competitive results but underperforms TrafficFormer in this task, with an F1 score of 0.522 compared to 0.543. Our approach surpasses the state-of-the-art TrafficFormer by 1.47% in F1 score, highlighting the efficacy of contrast learning in enhancing feature discriminability across traffic categories. Among few-shot methods, FC-Net, MetaMRE, and FS-MTC exhibit poor performance (F1 scores below 0.370, approximately 49.32% lower than ours), highlighting their limited capacity to learn discriminative features from small datasets when dealing with large, similar traffic categories.

• **EMC Task.** The results for the EMC task are shown in the second column group of Table V. In this task, the pre-training based methods are also superior to those of ML-based and DL-based ones. PERT still performs worst in all pre-training methods. TrafficLLM performs strongly, achieving an F1 score of 0.982, close to TrafficFormer's 0.983, benefiting from its LLM-based representation learning for malware traffic patterns. Our approach achieves marginal improvements of 0.10% across all metrics compared to ET-BERT but lags behind TrafficFormer by 0.81%, 0.91%, 1.02%, and 0.91% in accuracy, precision, recall, and F1 score, respectively. However, with 50% of training instances ($K = 50$) incorporated into our contrastive model, performance aligns with TrafficFormer, and with 100% incorporation, it exceeds TrafficFormer by 1.12%, 1.02%, 0.91%, and 0.91% in the respective metrics. The strong performance of baselines may be attributed to unencrypted traffic in the malware dataset, providing additional plaintext information. Among few-shot methods, FC-Net and

TABLE V
THE RESULTS OF DIFFERENT APPROACHES ON GEAC, EMC, ETCV, EACT AND EAC-1.3 TASKS UNDER CLOSED-WORLD SETTING

Tasks Approaches	GEAC				EMC				ETCV				EACT				EAC-1.3			
	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
Appscanner [42]	0.416	0.490	0.354	0.395	0.894	0.898	0.897	0.889	0.673	0.610	0.621	0.612	0.672	0.468	0.442	0.445	0.616	0.624	0.633	0.620
BIND [32]	0.303	0.279	0.271	0.252	0.846	0.868	0.838	0.839	0.715	0.637	0.632	0.633	0.718	0.460	0.452	0.451	0.796	0.781	0.745	0.751
FS-Net [26]	0.431	0.321	0.307	0.301	0.885	0.885	0.892	0.884	0.693	0.616	0.604	0.606	0.607	0.508	0.535	0.459	0.793	0.799	0.788	0.791
DeepPacket [29]	0.399	0.346	0.419	0.357	0.814	0.815	0.813	0.812	0.703	0.662	0.674	0.658	0.745	0.755	0.740	0.742	0.402	0.432	0.269	0.399
PERT [18]	0.612	0.524	0.508	0.512	0.967	0.970	0.965	0.966	0.786	0.716	0.721	0.705	0.768	0.442	0.445	0.435	0.801	0.716	0.767	0.723
ET-BERT [25]	0.625	0.545	0.516	0.516	0.970	0.974	0.972	0.973	0.871	0.812	0.828	0.814	0.831	0.556	0.645	0.589	0.812	0.799	0.789	0.788
TrafficFormer [54]	0.667	0.570	0.545	0.543	0.982	0.984	0.983	0.983	0.876	0.858	0.825	0.836	0.922	0.915	0.909	0.911	0.848	0.837	0.835	0.834
TrafficLLM [6]	0.619	0.521	0.532	0.522	0.984	0.985	0.979	0.982	0.883	0.856	0.832	0.838	0.942	0.933	0.910	0.928	0.816	0.813	0.795	0.801
FC-Net [46]	0.416	0.352	0.301	0.313	0.877	0.852	0.846	0.848	0.695	0.699	0.598	0.636	0.601	0.511	0.565	0.518	0.556	0.522	0.493	0.499
MetaMRE [48]	0.424	0.391	0.322	0.341	0.886	0.881	0.837	0.853	0.761	0.789	0.626	0.703	0.666	0.603	0.507	0.524	0.588	0.571	0.555	0.559
FS-MTC [49]	0.431	0.387	0.368	0.369	0.922	0.928	0.879	0.904	0.744	0.749	0.602	0.661	0.802	0.778	0.738	0.757	0.592	0.547	0.583	0.568
Ours	0.653	0.582	0.549	0.551	0.974	0.975	0.973	0.974	0.882	0.861	0.827	0.839	0.945	0.946	0.939	0.940	0.818	0.815	0.811	0.812
Ours:100-shot	0.708	0.645	0.601	0.606	0.993	0.994	0.992	0.992	0.926	0.919	0.910	0.912	0.989	0.986	0.979	0.980	0.876	0.863	0.849	0.852

MetaMRE achieve performance comparable to ML- and DL-based approaches but remain inferior to pre-training-based methods. FS-MTC, benefiting from preliminary knowledge derived from auxiliary dataset, achieves a higher F1 score than FC-Net and MetaMRE, yet still trails our method by 7.74%.

• **ETCV Task.** The results for the ETCV task are shown in the third column group of Table V, from where we can see that the classification performance of ML-based methods and DL-based methods is comparable and consistently inferior to that of the pre-training based methods. Among the baselines, TrafficLLM achieves $F1 = 0.838$ and TrafficFormer achieves 0.836, both of which are comparable to our method ($F1 = 0.839$). Nevertheless, with 100-shot data, the $F1$ score of our method rises to 0.912, which surpasses TrafficLLM and TrafficFormer by 8.83% and 9.09% respectively. Few-shot methods such as MetaMRE ($F1 = 0.703$) and FS-MTC ($F1 = 0.661$) remain notably below pre-training based methods (i.e., 19.34% lower than ours), confirming the importance of large-scale representation learning for imbalanced datasets like ETCV task. In addition, our contrast learning strategy further mitigates the problem of class imbalance by emphasizing instance similarity in the embedding space, improving representation of minority classes.

• **EACT Task.** The results for the EACT task, shown in the fourth column group of Table V, highlight poor performance across most baselines except TrafficFormer and TrafficLLM, due to Tor’s multi-layer encryption and routing, which obscure consistent patterns. Traffic features like packet size and timing, relied upon by ML-based methods, are rendered ineffective. TrafficFormer applies a traffic data augmentation method to reduce the model’s reliance on irrelevant information and enrich the training instances, which contributes a lot to the classification performance. TrafficLLM excels with an $F1$ score of 0.928, outperforming other baselines, thanks to its LLM-driven contextual representation for anonymized traffic. Our approach achieves 0.940, improving over TrafficFormer and TrafficLLM by 3.18% and 1.29% respectively. With 100-shot instances, our method further rises to 0.980, which is 6.95% higher than TrafficFormer and 5.60% higher than TrafficLLM. In contrast, few-shot baselines (e.g., MetaMRE 0.524, FC-Net 0.518) lag significantly, highlighting the advantage of pre-training combined with contrast learning under heavy obfuscation conditions.

• **EAC-1.3 Task.** The results for the EAC-1.3 task are shown in the rightmost part in Table V. Our approach performs better than all ML-based and DL-based methods. It improves by 8.12% and 2.65% in $F1$ score over the best ML-based method BIND and DL-based method FS-Net respectively. Among pre-training methods, we surpass PERT and ET-BERT with gains of 8.9% and 2.4% in $F1$ score, respectively. TrafficLLM ($F1 = 0.801$) performs competitively but falls short of TrafficFormer ($F1 = 0.834$) by 3.3%, while our method narrows this gap, trailing TrafficFormer by 2.2%. With full training data (100-shot), our approach further excels, surpassing TrafficFormer by 2.16% in $F1$ score. The slight underperformance under 20-shot conditions is justified by the practical costs of traffic labeling. Among few-shot methods, they exhibit significant performance gaps. While FS-MTC, benefiting from auxiliary dataset, achieves the best results, it still underperforms our method by 24.4%, highlighting the inherent limitations of existing few-shot approaches in handling large volumes of TLS 1.3 traffic—where they struggle to learn sufficiently discriminative features.

D. Comparison Under Open-World Setting

We select eight baselines for the comparison in open-world setting. Among these baselines, there are three ML-based methods: Whisper [12], FOSS [52], ProGraph [23], three DL-based methods: Prototype [50], K-LND [7], Trident [53], and two pre-training based methods: OWCP [24] and TrafficGPT [15]. It is worth noting that Whisper [12] does not possess multi-class classification capabilities. However, given its ability to detect unknown categories, it is still included as a comparative baseline. All evaluation metrics of Whisper are reported under the binary classification setting of known categories versus novel categories. We train all baseline models on the entire training set for each task to show their best performance. The experimental results are shown in Tables VI. Whisper demonstrates strengths in detecting unknown categories with $F1$ scores ranging from 0.477 to 0.903 across tasks, leveraging frequency domain analysis for robust realtime detection, but its lack of multi-class support limits its applicability in complex open-world scenarios. The following analysis will primarily focus on comparisons with other baselines in the multi-class context.

TABLE VI
THE RESULTS OF DIFFERENT APPROACHES ON GEAC, EMC, ETCV, EACT AND EAC-1.3 TASKS UNDER OPEN-WORLD SETTING

Tasks	GEAC				EMC				ETCV				EACT				EAC-1.3			
Approaches	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
Whisper [12]	0.502	0.488	0.475	0.477	0.911	0.914	0.896	0.903	0.812	0.806	0.798	0.801	0.867	0.858	0.836	0.842	0.762	0.728	0.746	0.733
FOSS [52]	0.393	0.344	0.305	0.311	0.798	0.807	0.754	0.766	0.615	0.606	0.589	0.595	0.686	0.701	0.637	0.645	0.494	0.488	0.465	0.469
ProGraph [23]	0.425	0.387	0.327	0.332	0.801	0.783	0.772	0.774	0.621	0.598	0.642	0.616	0.688	0.743	0.628	0.672	0.534	0.548	0.472	0.503
Prototype [50]	0.453	0.392	0.344	0.359	0.821	0.833	0.826	0.828	0.777	0.782	0.756	0.766	0.801	0.789	0.703	0.718	0.622	0.613	0.591	0.595
k-LND [7]	0.389	0.321	0.288	0.294	0.810	0.819	0.813	0.814	0.723	0.716	0.701	0.707	0.705	0.664	0.622	0.631	0.511	0.498	0.486	0.488
Trident [53]	0.456	0.481	0.395	0.406	0.838	0.845	0.827	0.832	0.762	0.771	0.750	0.758	0.796	0.753	0.807	0.774	0.668	0.671	0.654	0.658
OWCP [24]	0.574	0.504	0.435	0.462	0.867	0.872	0.869	0.870	0.824	0.801	0.765	0.771	0.848	0.827	0.816	0.821	0.635	0.652	0.598	0.601
TrafficGPT [15]	0.604	0.594	0.494	0.528	0.902	0.909	0.898	0.901	0.832	0.815	0.801	0.808	0.819	0.816	0.805	0.808	0.742	0.715	0.733	0.719
Ours	0.619	0.579	0.538	0.545	0.922	0.925	0.920	0.921	0.831	0.819	0.808	0.812	0.905	0.908	0.911	0.903	0.751	0.718	0.739	0.724
Ours:100-shot	0.683	0.612	0.582	0.590	0.962	0.959	0.963	0.959	0.905	0.900	0.861	0.873	0.957	0.951	0.961	0.952	0.829	0.831	0.811	0.819

• **GEAC Task (open).** The results for the GEAC task are shown in the leftmost part of Table VI. The pre-training based methods demonstrate better results compared to ML-based and DL-based methods. This superiority suggests the necessity of obtaining robust feature representations from pre-trained models for the open-world classification task. For the pre-training based methods, our approach improves 7.84%, 14.88%, 23.68%, 17.97% in terms of accuracy, precision, recall and F1 score over OWCP. The reason for the poor performance of OWCP is that it synthesizes new instances that around the boundaries of known classes and use them as the training instances from novel class, but these synthesized instances may not conform to the actual novel class distribution. In contrast, we make no assumptions about the distribution of new classes; instead, we focus on learning more discriminative features based on the available instances from the known classes. Our approach perform better than TrafficGPT in terms of accuracy (+ 2.48%), recall (+ 8.91%) and F1 score (+ 3.22%) but worse in precision (-2.53%). We attribute this to the insufficient training instances for contrast learning, as when 100% training instances are used ($K = 100$), the precision will be 3.03% higher than TrafficGPT.

• **EMC Task (open).** The results for the EMC task are shown in the second column group of Table VI. Our approach achieves the best performance across all evaluation metrics, demonstrating its effectiveness in this domain. Specifically, it outperforms the best-performing DL-based method Trident by 10.69% in F1 score and surpasses the top pre-training based method TrafficGPT by 2.22% in F1 score. These substantial improvements highlight the robustness and adaptability of our approach in handling the challenges associated with encrypted malware classification. The superior performance indicates that our method effectively captures discriminative features, enabling more accurate detection of known and novel malware. This underscores the potential of our approach as a valuable tool for real-world malware detection tasks.

• **ETCV Task (open).** The results for the ETCV task are shown in the third column group of Table VI. In this task, pre-training based methods demonstrate superior performance compared to the ML- and DL-based approaches. Specifically, our approach achieves improvements of 31.82%, 6.00% and 0.50% in F1 score over the state-of-the-art ML-based method Prograph, DL-based method Prototype and the pre-training based method TrafficGPT respectively. Addressing the data imbalance challenge inherent to this task,

our approach effectively mitigates the issue by: (1) capturing byte-level dependencies within BURSTs as well as the correlations between BURSTs, and (2) learning a similarity metric rather than directly searching for class boundaries. While our approach records a slightly lower accuracy (-0.12%) compared to TrafficGPT, it is important to note that TrafficGPT relies on dataset-specific fine-tuning in a closed-world setting to extract features prior to open-world classification. This process is less practical in real-world scenarios. By contrast, the generic feature representations derived from the pre-trained model in our approach are dataset-independent, making it more applicable to real-world situations and better aligned with practical requirements.

• **EACT Task (open).** The results for the EACT task are shown in the fourth column group of Table VI. Our approach achieves consistent improvements across all evaluation metrics, surpassing the existing best results obtained by OWCP by 6.72%, 9.79%, 11.64%, 9.99% in accuracy, precision, recall and F1 score respectively. This demonstrates the effectiveness of our method in addressing the challenges posed by multi-layer encrypted traffic. Such traffic introduces significant difficulties due to the obfuscation and encryption mechanisms inherent in the Tor protocol. OWCP tackles this challenge by firstly conducting contrastive pre-training to extract discriminative features and then synthesizing flows of novel class in the feature space to enrich the training instances. In contrast, we design explicit pre-training tasks to learn robust and generic feature representations that are better suited for encrypted traffic. These features are further refined and enhanced through a contrastive learning strategy, leading to improved discriminative capability. This combination enables our approach to handle the complexities of encrypted Tor traffic and perform open-world Tor traffic classification effectively.

• **EAC-1.3 Task (open).** The results for the EAC-1.3 task, presented in the rightmost part of Table VI, reveal that while pre-training methods typically lead, the DL-based method Trident surpasses the pre-training method OWCP by 9.48% in F1 score, benefiting from its decoupled one-class learning framework for fine-grained detection. Nevertheless, Trident still trails our approach by 10.03% in F1 score. Our approach outperforms the best baseline, TrafficGPT, achieving improvements of 1.21%, 0.42%, 0.82%, and 0.69% in accuracy, precision, recall, and F1 score, respectively. These results suggest that our method's ability to learn generic yet

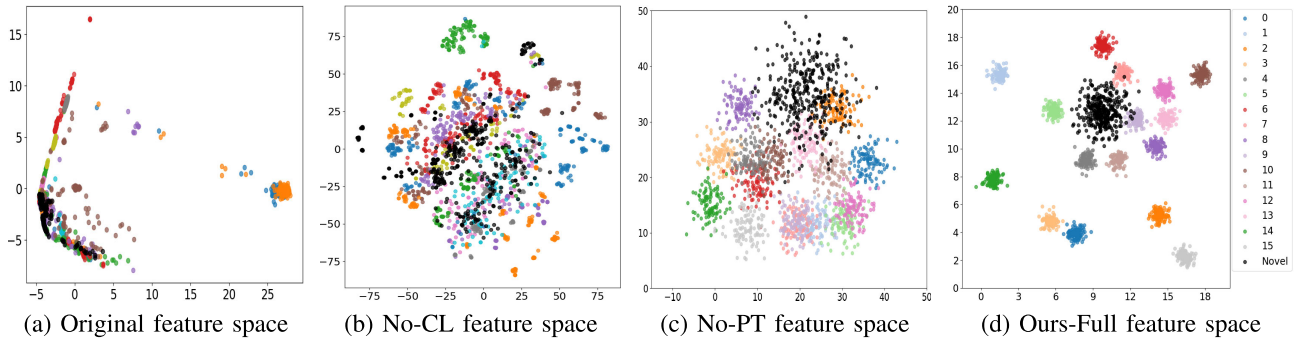


Fig. 4. The visualization for traffic flows on EMC task in different feature space.

TABLE VII
ABLATION RESULTS ON EMC TASK

Settings Approaches	closed-world				open-world			
	AC	PR	RC	F1	AC	PR	RC	F1
No-PT	0.762	0.768	0.735	0.744	0.724	0.731	0.723	0.725
No-CL	0.532	0.575	0.498	0.521	0.455	0.482	0.437	0.451
ET-BERT+CL	0.968	0.965	0.963	0.964	0.918	0.914	0.912	0.913
TrafficFormer+CL	0.975	0.977	0.971	0.974	0.924	0.923	0.921	0.922
Ours	0.974	0.975	0.973	0.974	0.922	0.925	0.920	0.921

discriminative features through task-specific pre-training, combined with contrast learning strategies, allows it to generalize better to complex and highly encrypted traffic. Furthermore, the superiority of our approach highlights its practical value for handling real-world encrypted application classification tasks, especially under the constraints posed by modern encryption protocols such as TLS 1.3.

E. Ablation Study

To address RQ3 and RQ4, we analyze the contributions of pre-training and contrast learning, and further examine the effectiveness of our two pre-training tasks as well as the impact of model capacity.

1) *Impact of Components*: To examine the impact of pre-training and contrast learning components, we set up four baselines as follows: (1) **NO Pre-Training (NO-PT)**: raw bytes of the first 4 packets (32 bytes each) after sanitizing biased fields are used for contrast learning. (2) **No Contrast Learning (NO-CL)**: applying our open-world classification on features from the pre-trained model. (3) **ET-BERT+CL**: replacing our pre-training model with ET-BERT. (4) **TrafficFormer+CL**: replacing our pre-training model with TrafficFormer.

Results on the EMC task (Table VII) show that methods combining both pre-training and contrast learning consistently outperform those using only one component, confirming their complementary roles. NO-CL yields the worst performance, indicating that contrast learning is crucial for obtaining discriminative representations. NO-PT performs better than NO-CL but still lags behind other baselines, showing that pre-training benefits generic feature extraction and facilitates subsequent discriminative learning. Compared with ET-BERT+CL, our approach improves F1 score by 1.04% and 0.88% in closed- and open-world settings, respectively. ET-BERT only considers intra-BURST correlations, whereas

our MBM and BCP jointly capture both intra- and inter-BURST relationships. Our method also matches TrafficFormer+CL despite TrafficFormer using a more complex SODF task, demonstrating that our simpler MBM+BCP design is both efficient and effective.

To provide a more intuitive comparison, we visualize the distributions of traffic flows on EMC task by projecting them onto a two-dimensional feature space using t-SNE. As illustrated in Fig. 4, the plots have instances from 16 known classes, indicated from 0 to 15 in different colors, and a set of instances from novel class (i.e., the remaining 4 categories of EMC task) indicated in black color. Fig. 4a shows the distribution of raw traffic. This is achieved by selecting the first 4 packets from each flow and the first 32 bytes from each packet. The resulting 4×32 -dimensional feature vector is then reduced to two dimensions after sanitizing biased fields. Meanwhile, Fig. 4b illustrates the distribution of traffic flows output by NO-CL, Fig. 4c is the output of NO-PT and Fig. 4d displays the output of our complete method. Clearly, the instances are completely inseparable in the original feature space. After through the pre-training model, the instance distribution is more dispersed, but the separability is still poor. Directly apply contrast learning strategy on the raw traffic can obtain features with a certain degree of discriminability compared to No-CL or original input, but the instance clusters from different classes still have a high degree of overlap. In contrast, the output of our complete method has more distinct groups of instance clusters for different classes compared to both No-CL and No-PT and instances from novel classes are well separated from other clusters containing known class instances. This also indicates that both the pre-training and contrast learning component play crucial roles in our approach and complement each other. In particular, the pre-training component help extract robust initial feature representations and the contrast learning component make these feature representations more discriminative to adapt to the open-world encrypted traffic classification problem.

2) *Effectiveness of Pre-Training Tasks*: Furthermore, to verify the effectiveness of our proposed pre-training tasks (MBM and BCP) in learning generic and robust feature representations, we perform an ablation study using adversarial samples generated via the Fast Gradient Sign Method (FGSM) [16]. FGSM introduces controlled perturbations to the original traffic representations, mimicking real-world noise such as packet error or network jitter, thereby enabling an evaluation of the learned features' resilience.

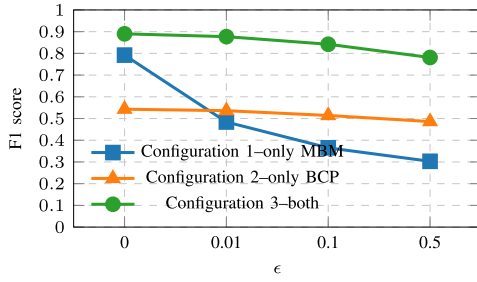


Fig. 5. F1 score variation with increasing ϵ on ETCV task.

Adversarial samples are generated on the test set using the FGSM formulation:

$$x' = x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y)) \quad (16)$$

where x represents the embedded representation of a BURST—a sequence of tokenized traffic units derived from raw flows via our preprocessing pipeline. Here, ϵ controls the perturbation magnitude (evaluated at 0, 0.01, 0.1, and 0.5), L is the loss function, and sign denotes the sign function. The perturbations are applied to the token embeddings of BURSTs, which are processed by the BERT-based model. We compare three configurations on the ETCV task, where models are fine-tuned with a softmax classifier and then tested on the perturbed test set. The three configurations are as follows:

- *Configuration 1*: Pre-training with only MBM, which reconstructs masked byte sequences within a BURST to learn generic structural patterns.
- *Configuration 2*: Pre-training with only BCP, which predicts contextual relationships between BURSTs to enhance robustness.
- *Configuration 3*: Pre-training with both MBM and BCP (our proposed model).

Fig. 5 illustrates the variation in F1 scores across three configurations under different perturbation magnitudes ϵ . Without perturbation, *Configuration 1* exhibits an F1 score 12.3% lower than *Configuration 3*, and its performance deteriorates sharply with minor perturbation ($\epsilon = 0.01$). Under significant perturbation, the F1 score drops to 0.392, suggesting that while the MBM task facilitates effective feature learning, features derived solely from MBM lack robustness. Conversely, *Configuration 2* yields the lowest F1 score (below 0.543) even without perturbation, indicating that learning contextual relationships between BURSTs alone does not yield generic features. In contrast, *Configuration 3* shows minimal F1 score degradation (1.43% drop) with minor perturbation. As ϵ increases, the F1 score remains relatively stable, retaining a value above 0.780 even at $\epsilon = 0.5$, underscoring the synergistic role of MBM and BCP pre-training tasks in ensuring the acquisition of generic and robust feature representations.

In addition, to further evaluate whether the generic and robust features stem from increased model capacity, we configure the pre-training model with varying architectures as follows:

- *Cap. 1*: 2 Transformer block with model dimension 32.
- *Cap. 2*: 12 Transformer block with model dimension 32.
- *Cap. 3*: 2 Transformer block with model dimension 128.
- *Cap. 4*: 12 Transformer block with model dimension 128.
- *Cap. 5*: 24 Transformer block with model dimension 256.

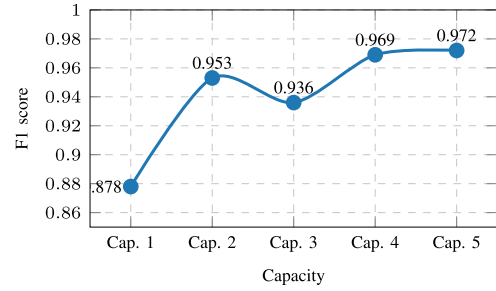


Fig. 6. F1 score variation on EMC task.

The model capacity increases progressively from *Cap. 1* to *Cap. 5*. Fig. 6 depicts the variation in F1 scores across these five configurations on the EMC task, where fine-tuning is performed using a softmax classifier following pre-training. The results indicate that the F1 score improvement plateaus from *Cap. 2* to *Cap. 5*. Notably, despite *Cap. 3* having a larger model capacity than *Cap. 2*, its F1 score decreases by 1.82%. Furthermore, comparisons between *Cap. 2* and *Cap. 4*, and between *Cap. 4* and *Cap. 5*, reveal that merely increasing model width or depth yields marginal gains, with F1 score improvements of only 1.68% and 0.31%, respectively. These findings underscore that simply augmenting model capacity does not significantly contribute to acquiring generic and robust feature representations; rather, such representations are the result of the synergistic interplay between our pre-training tasks MBM and BCP.

F. Overhead Analysis

In this section, we first give the model parameter quantity and then perform an overhead analysis of our approach, measuring training time (s/100 batches), inference time (ms/flow) and GPU memory usage during both training and testing. The pre-training model parameter mainly consists of four parts: token embeddings, positional embeddings, Transformer encoders and output layers. According to our implementation settings, the approximate number of parameters is as follows:

$$\begin{aligned}
 \#Param1 &\approx \underbrace{V \times D}_{\text{token emb}} + \underbrace{\hat{L} \times D}_{\text{pos emb}} \\
 &\quad + \underbrace{N \times (12D^2 + 13D)}_{\text{12-layer encoder}} + \underbrace{(V+1) \times D}_{\text{output}} \\
 &= 65,536 \times 128 + 512 \times 128 \\
 &\quad + 12 \times (12 \times 128^2 + 13 \times 128) \\
 &\quad + (65,536 + 1) \times 128 \\
 &\approx \mathbf{19.3M} \text{ parameters.} \quad (17)
 \end{aligned}$$

where V denotes the vocabulary size; D denotes the model dimension; $\hat{L}$ denotes the maximum sequence length; N denotes the number of Transformer blocks. Similarly, the model of contrast learning mainly comes from two parts: the convolutional layers in the residual blocks (including their shortcut connections), and the fully connected layers. The approximate number of parameters is calculated as follows:

$$\#Param2 \approx \underbrace{\sum_{i=1}^8 (3 \times C_{in}^{(i)} C_{out}^{(i)} + C_{out}^{(i)})}_{\text{1D conv (8 layers)}}$$

TABLE VIII
OVERHEAD STATISTICS OF OUR APPROACH

	Time Overhead		GPU Memory	
	Training (s/100 batches)	Inference (ms/flow)	Training (MB)	Inference (MB)
Pre-training	5.31	1.12	9719	3773
Contrast Learning	2.64	0.05	46	12
Total	7.95	1.17	9765	3785

$$\begin{aligned}
& + \underbrace{\sum_{j=1}^4 (1 \times C_{in}^{(j)} C_{out}^{(j)} + C_{out}^{(j)})}_{1 \times 1 \text{ shortcut (4 layers)}} \\
& + \underbrace{\sum_{k=1}^3 (D_{in}^{(k)} \cdot D_{out}^{(k)} + D_{out}^{(k)})}_{\text{fully connected layers (fc1-fc3)}} \\
& = \underbrace{59,968}_{\text{conv layers}} + \underbrace{6,656}_{\text{shortcut layers}} + \underbrace{198,914}_{\text{FC layers}} \\
& \approx \mathbf{265.5K} \text{ parameters} \tag{18}
\end{aligned}$$

where $C_{in}^{(i)}$ and $C_{out}^{(i)}$ denote the input and output channels of the i -th convolution layer; $C_{in}^{(j)}$ and $C_{out}^{(j)}$ denote the input and output channels of the j -th shortcut convolution; $D_{in}^{(k)}$ and $D_{out}^{(k)}$ denote the input and output dimensions of the k -th fully connected layer.

The overhead statistics based on the above model capacity are shown in Table VIII. To ensure accurate measurements, we restricted the system such that when a model task was executed, it remained the only CPU- and GPU-intensive job on the machine. Training time was calculated by recording the total time required to process the entire training set, dividing by the total number of batches, and then multiplying by 100. As we can see from the first column of Table VIII, our method takes 5.31s and 2.64s respectively in the pre-training and contrast learning stage, totally 7.95s. To calculate the inference time, we recorded the total time taken by the model to infer the entire test set and then divided by the number of test flows. Our approach takes 1.12 ms and 0.05 ms per flow respectively during the pre-training and contrast learning stage, totally 1.17 ms, which are shown in the second column of Table VIII. The last two columns show the GPU memory usage during both training and testing. The pre-training model's GPU memory consumption during training is relatively high (9719MB) due to the need to learn attention-based token embedding representations for tokens derived from BURSTs, which involves extensive forward and backward calculations across Transformer blocks and generates a large number of intermediate variables. However, during testing, the GPU memory usage drops to 3773MB since gradient calculations and back propagation are not required. In comparison, for contrast learning, the GPU memories are low during both training (46MB) and testing (12MB) phase, because only the small contrast embedding network requires gradient computation and parameter updates, which significantly reduces the memory footprint.

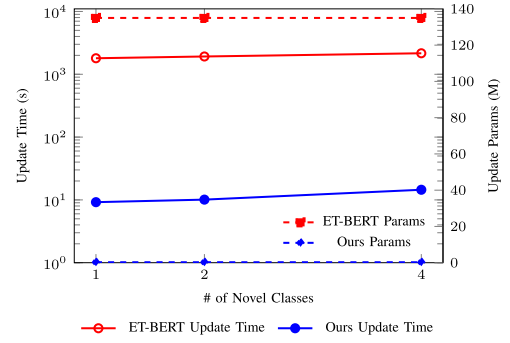


Fig. 7. Comparison of model update cost during emergence of novel classes on EMC task.

G. Operational Considerations

In this section, we discuss the practical considerations involved in deploying our approach in a production environment. Two primary challenges arise in operational settings: i) the computational cost of pre-training, and ii) the ability to efficiently update the model when new categories emerge.

One-time pre-training cost. The pre-training stage is designed to be a one-time offline process that captures generalizable patterns across a wide range of traffic types. Retraining the pre-trained model is typically unnecessary unless the underlying traffic distribution undergoes substantial and persistent shifts, such as the introduction of new encryption standards, major application-layer protocol redesigns, or fundamental changes in traffic composition. In such cases, updating the pre-trained model with newly collected unlabeled traffic restores representational relevance while keeping long-term maintenance manageable.

Fast and lightweight model updates. The second operational challenge concerns incremental updates when novel traffic classes appear. Traditional softmax-based models, such as ET-BERT, suffer from *architectural coupling*, where the output layer is tied to a fixed number of classes. Consequently, the emergence of a novel class necessitates a structural modification of the classification head followed by a complete fine-tuning of the entire model, a process that is both disruptive and time-consuming (i.e., the update cost is orders of magnitude higher than ours as shown in Fig. 7). In contrast, our approach utilizes a gradient-isolated update strategy. Since the pre-trained backbone is frozen, only the lightweight contrastive module requires updating. As illustrated in Fig. 7, our approach achieves “plug-and-play” adaptation, completing the model refresh in less than 15 seconds with a negligible memory footprint (<50 MB), regardless of the number of novel classes. These findings confirm that our framework is substantially more scalable in real-world environments where new traffic categories emerge frequently and rapid adaptation is essential.

Recommended deployment workflow. Based on the above considerations, our approach may be deployed in practice following the workflow: i) collect unlabeled traffic data from the target environment and conduct one-time pre-training; ii) label a small number of samples for the downstream task and perform contrast learning; iii) carry out online classification using the trained model; iv) maintain a candidate pool of traffic instances predicted as novel classes; and v) periodically

label a small subset from the candidate pool to refresh the contrastive module.

VI. DISCUSSION

In this section, we discuss some limitations of our approach and its potential impact on further research in this field. i) BURST division. Our BURST division employs a primary criterion: grouping consecutive packets in the same direction. However, we acknowledge that network attributes such as inter-packet delays and idle times can influence the logical segmentation of traffic. Even within the same direction, prolonged intervals between packets may signify distinct application-layer operations, such as separate resource requests, which could justify further segmentation. We note that the BCP task is designed to learn contextual relationships between BURSTs. This capability helps mitigate the impact of imperfect segmentation by enabling the model to discern whether BURSTs belong to the same flow or represent unrelated operations, thereby preserving structural characteristics at a higher contextual level. ii) Flow-level classification. We learn robust feature representation for each flow and perform classification task at single-flow level. We have not taken the multi-flow relations into consideration, which may degrade the performance under multi-flow scenarios (e.g. P2P network usually generate multi-flows to transmit files between peers). Meanwhile, we cannot perform fine-grained classification at the packet level as well. iii) Fairness of comparison. Our approach relies on a pre-training stage, whereas several baselines do not. We acknowledge that this asymmetry may raise concerns regarding fairness. However, to the best of our knowledge, no existing method simultaneously supports pre-training, few-shot learning, and novel class detection. As a result, it is difficult to compare against a single unified baseline that covers all three capabilities. Instead, we selected multiple representative methods, each corresponding to a different subset of these capabilities (pre-training based classifiers, few-shot learners, and novelty detectors). This design ensures that every dimension of our method is evaluated against the strongest available alternatives. Future research may explore unified benchmarks that incorporate pre-training for all candidate methods, enabling more strictly controlled comparisons.

VII. CONCLUSION

In this paper, we propose an efficient open-world encrypted traffic classification approach, which achieves few-shot classification by combining pre-training techniques with contrast learning. In the pre-training stage, we design two pre-training tasks: MBM task and BCP task, one learns the byte dependencies within BURSTs and the other learns the relationships between BURSTs, together they guarantee the robust initial feature representations of traffic flows. In the contrast learning stage, a discriminative feature space is learned based on the customized contrast learning strategy. The characteristic of intra-class compactness and inter-class separability within this feature space allows us to employ a distance-based metric for classifying known categories and detecting novel ones. We evaluate our approach on five encrypted traffic classification tasks and the results demonstrate that our approach achieves consistently better performance than baseline studies in both closed- and open-world settings.

REFERENCES

- [1] G. Aceto, D. Ciunzio, A. Montieri, and A. Pescapé, "DISTILLER: Encrypted traffic classification via multimodal multitask deep learning," *J. Netw. Comput. Appl.*, vols. 183–184, Jun. 2021, Art. no. 102985.
- [2] H. Bao, L. Dong, S. Piao, and F. Wei, "BEiT: BERT pre-training of image transformers," 2021, *arXiv:2106.08254*.
- [3] P. L. Bartlett and S. Mendelson, "Rademacher and Gaussian complexities: Risk bounds and structural results," *J. Mach. Learn. Res.*, vol. 3, pp. 463–482, Nov. 2002.
- [4] L. Bernaille, R. Teixeira, I. Akodkenou, A. Soule, and K. Salamatin, "Traffic classification on the fly," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 36, no. 2, pp. 23–26, Apr. 2006.
- [5] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [6] T. Cui et al., "TrafficLLM: Enhancing large language models for network traffic analysis with generic traffic representation," 2025, *arXiv:2504.04222*.
- [7] T. Dahanayaka, Y. Ginige, Y. Huang, G. Jourjon, and S. Seneviratne, "Robust open-set classification for encrypted traffic fingerprinting," *Comput. Netw.*, vol. 236, Nov. 2023, Art. no. 109991.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [9] I. Din, N. A. Saqib, and A. Baig, "Passive analysis of web traffic characteristics for estimating quality of experience," in *Proc. IEEE Globecom Workshops*, Nov. 2008, pp. 1–5.
- [10] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [11] Z. M. Fadlullah, T. Taleb, A. V. Vasilakos, M. Guizani, and N. Kato, "DTRAB: Combating against attacks on encrypted protocols through traffic-feature analysis," *IEEE/ACM Trans. Netw.*, vol. 18, no. 4, pp. 1234–1247, Aug. 2010.
- [12] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.
- [13] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," 2021, *arXiv:2104.08821*.
- [14] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.
- [15] Y. Ginige, T. Dahanayaka, and S. Seneviratne, "TrafficGPT: An LLM approach for open-set encrypted traffic classification," in *Proc. 19th Asian Internet Eng. Conf.*, Aug. 2024, pp. 26–35.
- [16] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [17] J.-B. Grill et al., "Bootstrap your own latent—a new approach to self-supervised learning," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 21271–21284.
- [18] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.
- [19] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9729–9738.
- [20] T.-L. Huoh, Y. Luo, P. Li, and T. Zhang, "Flow-based encrypted network traffic classification with graph neural networks," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1224–1237, Jun. 2023.
- [21] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," 2019, *arXiv:1909.11942*.
- [22] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Porto, Portugal, Feb. 2017, pp. 253–262.
- [23] W. Li, X.-Y. Zhang, H. Bao, H. Shi, and Q. Wang, "ProGraph: Robust network traffic identification with graph propagation," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1385–1399, Jun. 2023.
- [24] X. Li, B. Feng, T. Zang, X. Xu, S. Zhao, and J. Ma, "Facing unknown: Open-world encrypted traffic classification based on contrastive pre-training," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2023, pp. 1255–1260.

- [25] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.
- [26] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.
- [27] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
- [28] Z. Liu, L. Cai, L. Zhao, A. Yu, and D. Meng, "Towards open world traffic classification," in *Proc. Inf. Commun. Security: 23rd Int. Conf.*, Chongqing, China, Nov. 2021, pp. 331–347.
- [29] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.
- [30] T. Mitchell et al., "Never-ending learning," *Commun. ACM*, vol. 61, no. 5, pp. 103–115, 2018.
- [31] A. W. Moore and D. Zuev, "Internet traffic classification using Bayesian analysis techniques," in *Proc. ACM SIGMETRICS Int. Conf. Meas. Model. Comput. Syst.*, Jun. 2005, pp. 50–60.
- [32] K. Al-Naami et al., "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 177–188.
- [33] A. Panchenko et al., "Website fingerprinting at internet scale," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–15.
- [34] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, "Website fingerprinting in onion routing based anonymization networks," in *Proc. 10th Annu. ACM workshop Privacy Electron. Soc.*, Oct. 2011, pp. 103–114.
- [35] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [36] M. Roughan, S. Sen, O. Spatscheck, and N. Duffield, "Class-of-service mapping for QoS: A statistical signature-based approach to IP traffic classification," in *Proc. 4th ACM SIGCOMM Conf. Internet Meas.*, Oct. 2004, pp. 135–148.
- [37] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.
- [38] T. Shapira and Y. Shavitt, "FlowPic: Encrypted internet traffic classification is as easy as image recognition," in *Proc. IEEE INFOCOM Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Apr. 2019, pp. 680–687.
- [39] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [40] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Toronto, ON, Canada, Oct. 2018, pp. 1928–1943.
- [41] Z. Song et al., "I² RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification," *IEEE Trans. Depend. Secure Comput.*, early access, 2023, doi: [10.1109/TDSC.2023.3245411](https://doi.org/10.1109/TDSC.2023.3245411).
- [42] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.
- [43] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.
- [44] A. Vaswani, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 1–11.
- [45] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.
- [46] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3540–3552, 2020.
- [47] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022.
- [48] C. Yang et al., "Few-shot encrypted traffic classification via multi-task representation enhanced meta-learning," *Comput. Netw.*, vol. 228, Jun. 2023, Art. no. 109731.
- [49] X. Zhang et al., "Enhanced few-shot malware traffic classification via integrating knowledge transfer with neural architecture search," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5245–5256, 2024.
- [50] L. Zhao, L. Cai, A. Yu, Z. Xu, and D. Meng, "Prototype-based malware traffic classification with novelty detection," in *Proc. Inf. Commun. Security: 21st Int. Conf.*, Beijing, China, Dec. 2020, pp. 3–17.
- [51] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.
- [52] Z. Zhao et al., "FOSS: Towards fine-grained unknown class detection against the open-set attack spectrum with variable legitimate traffic," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 3945–3960, Oct. 2024.
- [53] Z. Zhao, Z. Li, Z. Song, W. Li, and F. Zhang, "Trident: A universal framework for fine-grained and class-incremental unknown traffic detection," in *Proc. ACM Web Conf.*, May 2024, pp. 1608–1619.
- [54] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 1844–1860.



Lixin Zhao received the B.S. degree from Nankai University in 2013 and the Ph.D. degree in computer sciences from the University of Chinese Academy of Sciences in 2020. He is currently a Research Assistant with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include network security and data security.



Changhao Wu received the B.S. degree from Nankai University in 2022 and the M.S. degree in cyberspace security from the Institute of Information Engineering, Chinese Academy of Sciences, in 2025. His research interests include network security and intrusion detection.



Aimin Yu received the B.S. degree from Nankai University and the Ph.D. degree from the Institute of Software, Chinese Academy of Sciences, in 2011. He is currently a Professor with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include system security and trust computing.